



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Наташа Кашиковић

**Библиотека за аутентификацију
микросервисне комуникације
коришћењем метода Zero-Knowledge
Proofs**

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски рад
Аутор, АУ:	Наташа Кашиковић
Ментор, МН:	др Горан Савић, редовни професор
Наслов рада, НР:	Библиотека за аутентификацију микросервисне комуникације коришћењем метода Zero-Knowledge Proofs
Језик публикације, ЈП:	Српски / ћирилица
Језик извода, ЈИ:	Српски
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	АП Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, Трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/цитата/табела/слика/графика/листинга/прилога)	7/40/16/4/5/0/14/0
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Zero-Knowledge Proofs, Шнор протокол, Fiat-Shamir трансформација, аутентификација микросервиса, криптографски протоколи, Rust
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Рад се бави примјеном метода доказа без знања (енг. <i>Zero-Knowledge Proofs</i>) за аутентификацију комуникације између микросервиса. Циљ рада је развој библиотеке у програмском језику Rust која омогућава провјеру идентитета сервиса без директног преноса тајних података. Имплементиран је Schnorr протокол уз коришћење Fiat-Shamir трансформације, као и механизми заштите од поновног коришћења аутентификационих порука. Функционалност библиотеке демонстрирана је кроз апликацију засновану на микросервисној архитектури, која се састоји од сервиса за обраду наруџбина, управљање залихама и обраду плаћања. Резултати показују да се докази без знања могу примијенити као механизам за аутентификацију међусервисне комуникације уз очување тајности приватних кључева.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	
Чланови комисије, КО:	Председник:
	Члан:
	Члан, ментор:
	Потпис ментора

Accession number, ANO :			
Identification number, INO :			
Document type, DT :	Monographic publication		
Type of record, TR :	Textual printed material		
Contents code, CC :	Bachelor Thesis		
Author, AU :	Nataša Kašiković		
Mentor, MN :	Goran Savić, PhD, full professor		
Title, TI :	A Library for Microservice Communication Authentication Using Zero-Knowledge Proof Methods		
Language of text, LT :	Serbian		
Language of abstract, LA :	Serbian		
Country of publication, CP :	Republic of Serbia		
Locality of publication, LP :	Vojvodina		
Publication year, PY :	2026		
Publisher, PB :	Author's reprint		
Publication place, PP :	Novi Sad, Dositeja Obradovića sq. 6		
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/listings/appendixes)	7/40/16/4/5/0/14/0		
Scientific field, SF :	Electrical and computer engineering		
Scientific discipline, SD :	Applied computer science and informatics		
Subject/Key words, S/KW :	Zero-Knowledge Proofs, Schnorr protocol, Fiat-Shamir transformation, microservice authentication, cryptographic protocols, Rust		
UC			
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad, Serbia		
Note, N :			
Abstract, AB :	This thesis explores the application of Zero-Knowledge Proofs for authenticating communication between microservices. The aim of the thesis is to develop a library in the Rust programming language that enables service identity verification without the direct transmission of secret data. The Schnorr protocol is implemented using the Fiat-Shamir transformation, along with mechanisms for preventing the reuse of authentication messages. The functionality of the library is demonstrated through an application based on a microservice architecture consisting of services for order processing, inventory management, and payment processing. The results demonstrate that Zero-Knowledge Proofs can be applied as a mechanism for authenticating inter-service communication while preserving the confidentiality of private keys.		
Accepted by the Scientific Board on, ASB :			
Defended on, DE :			
Defended Board, DB :	President:		
	Member:		Menthor's sign
	Member, Mentor:		

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Наташа Кашиковић	Број индекса:	SV69/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Др. Горан Савић, редовни професор.		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Библиотека за аутентификацију микросервисне комуникације коришћењем метода Zero-Knowledge Proofs
--

ТЕКСТ ЗАДАТКА:

Извршити преглед основних концепата <i>Zero-Knowledge Proof</i> протокола и њихових примјена. Анализирати традиционалне приступе аутентификацији комуникације између микросервиса. Размотрити могућност примјене ZKP протокола као алтернативног приступа аутентификацији. Реализовати библиотеку за аутентификацију микросервисне комуникације засновану на Schnorr протоколу и Fiat-Shamir трансформацији. Интегрисати развијену библиотеку у демонстрациону микросервисну апликацију. Приказати њену примјену у процесу аутентификације комуникације између сервиса. Извршити функционално тестирање развијеног рјешења. Извршити мјерење перформанси генерисања и провјере доказа. Анализирати добијене резултате и размотрити предности и ограничења реализованог приступа.

Руководилац студијског програма:	Ментор рада:

Примерак за: ☐ - Студента; ☐ - Ментора
--

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Садржај

1	Увод.....	1
2	Теоријске основе <i>Zero-Knowledge Proof</i> протокола.....	2
2.1	Основни концепт <i>Zero-Knowledge Proof</i> протокола.....	2
2.2	Основне особине <i>Zero-Knowledge Proof</i> протокола	3
2.2.1	Потпуност.....	3
2.2.2	Поузданост.....	3
2.2.3	Нулто знање.....	3
2.3	Начин функционисања <i>Zero-Knowledge Proof</i> протокола	4
2.4	Интерактивни <i>ZKP</i>	5
2.5	Неинтерактивни <i>ZKP</i>	7
2.6	<i>Schnorr</i> протокол.....	9
2.7	Примјене <i>Zero-Knowledge Proof</i> протокола	10
2.7.1	Блокчејн и криптовалуте	10
2.7.2	Дигитални идентитет и <i>credentials</i>	10
2.7.3	Електронско гласање	10
2.8	Аутентификација сервиса у микросервисним системима	12
2.8.1	Традиционални приступи аутентификацији сервиса	12
2.8.2	Безбједносни ризици традиционалних приступа	13
2.8.3	<i>ZKP</i> као приступ аутентификацији сервиса	13
3	Пројектовање и имплементација <i>ZKP</i> библиотеке.....	14
3.1	Архитектура библиотеке	14
3.1.1	Унутрашња организација библиотеке.....	14
3.1.2	Јавни API библиотеке	15
3.2	Генерисање кључева	16
3.3	Формирање доказа	18
3.4	Провјера доказа.....	21
3.5	Провјера контекста захтјева и заштита од <i>replay</i> напада	23
3.6	Тестирање библиотеке.....	25
3.6.1	Тестови у оквиру <i>Schnorr crate</i>	25
3.6.2	Тестови у оквиру <i>Auth crate</i>	26
4	Примјена библиотеке у микросервисном окружењу	27
4.1	Архитектура демонстрационе апликације.....	27
4.2	Имплементација демонстрационе апликације.....	27
4.2.1	Имплементација <i>OrderService</i> сервиса	28

4.2.2	Имплементација <i>WarehouseService</i> сервиса.....	29
4.2.3	Имплементација <i>PaymentService</i> сервиса	29
4.3	Ток рада апликације	31
4.4	Интеграција ZKP библиотеке	32
4.5	Перформансе ZKP аутентификације	34
4.6	Поређење перформанси ZKP и традиционалних приступа аутентификацији	36
5	Закључак	38
6	Литература.....	39
7	Биографија.....	40

1 Увод

Развојем дистрибуираних информационих система и све већом примјеном микросервисне архитектуре, комуникација између сервиса постала је један од кључних аспеката развоја савремених софтверских система [1]. Умјесто монолитних апликација, функционалности се данас често имплементирају као скуп независних сервиса који међусобно размјењују податке путем рачунарске мреже. Такав приступ омогућава већу скалабилност, лакше одржавање и независно развијање појединих дијелова система, али истовремено уводи нове изазове у области безбједности [2].

Један од основних захтјева приликом комуникације између сервиса јесте поуздана аутентификација. Сервис који прима захтјев мора бити сигуран да захтјев заиста долази од овлашћеног сервиса. Истовремено треба спријечити неовлашћен приступ, злоупотребу идентитета и различите врсте напада. У пракси се за ову намјену најчешће користе API кључеви, приступни токени, TLS механизми засновани на дигиталним сертификатима и приступи засновани на *cloud-native* идентитетима сервиса [3], [4], [8], [9]. Иако су наведени механизми широко прихваћени, они се углавном заснивају на размјени или чувању тајних података, што представља потенцијални безбједносни ризик у случају њиховог компромитовања.

Развој савремених криптографских техника омогућио је примјену нових приступа аутентификацији код којих се тајни подаци не откривају током провјере. Значајно мјесто међу њима заузимају *Zero-Knowledge Proof* (ZKP) протоколи. Они омогућавају једном учеснику да другом докаже познавање одређене информације без њеног откривања [5]. У процесу аутентификације могу се примијенити тако да доказивач потврди посједовање тајне без њеног просљеђивања провјериоцу, чиме се смањује ризик од њеног откривања током комуникације [11].

Предмет овог рада јесте развој библиотеке за аутентификацију између микросервиса засноване на ZKP протоколима. Циљ библиотеке је да омогући доказивање идентитета сервиса без размјене тајних података, чиме се пружа алтернатива традиционалним приступима заснованим на API кључевима. Поред саме имплементације библиотеке, биће приказан и примјер њене интеграције у микросервисно окружење како би се демонстрирала њена практична примјена.

У другом поглављу биће представљене теоријске основе ZKP протокола, укључујући основне принципе рада, криптографске особине и најзначајније врсте ових протокола. Треће поглавље биће посвећено пројектовању и имплементацији развијене библиотеке, при чему ће бити описани њена архитектура, начин рада и кључне функционалности. У четвртом поглављу биће приказана интеграција библиотеке у примјер микросервисне апликације, као и анализа њеног рада. На крају рада биће изнесени закључци, размотрене предности развијеног рјешења и дати приједлози за његов даљи развој.

2 Теоријске основе *Zero-Knowledge Proof* протокола

У овом поглављу детаљније ћемо размотрити *Zero-Knowledge Proof* (ZKP) протоколе. Најприје ће бити објашњена основна идеја овог криптографског приступа, као и особине које један систем мора да испуни како би се могао сматрати *Zero-Knowledge Proof* системом. Након тога биће описан начин функционисања ових протокола, њихова подјела на интерактивне и неинтерактивне протоколе, као и неки од најзначајнијих приступа који се данас користе.

2.1 Основни концепт *Zero-Knowledge Proof* протокола

Развојем информационих система јавља се све већа потреба за механизмима који омогућавају провјеру одређених информација без њиховог директног откривања. Један од криптографских приступа који омогућава овакав начин провјере представљају *Zero-Knowledge Proof* протоколи. Основна идеја ових протокола јесте да један учесник може доказати другом учеснику истинитост одређене тврдње, а да при томе не открије никакве додатне информације осим чињенице да она важи.

У *Zero-Knowledge Proof* протоколу учествују двије стране: доказивач (енгл. *prover*) и провјерилац (енгл. *verifier*). Доказивач посједује одређену тајну информацију и жели да увјери провјериоца да је одговарајућа тврдња тачна. При томе, сама тајна информација не треба да буде откривена провјериоцу. Провјерилац на основу података које добија током извршавања протокола одлучује да ли ће прихватити или одбити доказ.

На примјер, корисник може жељети да докаже да познаје одређену тајну вриједност без њеног директног слања другој страни. Код традиционалног приступа било би потребно доставити податак на основу којег друга страна може извршити провјеру. Примјеном *Zero-Knowledge Proof* протокола могуће је формирати доказ који потврђује посједовање одговарајуће информације, а да сама информација остане скривена. Управо ова особина чини ZKP погодним за системе у којима је потребно истовремено обезбиједити провјеру исправности и очување приватности.

Да би се одређени протокол могао сматрати ZKP протоколом, потребно је да задовољи три основне особине:

- Потпуност (енгл. *completeness*),
- Поузданост (енгл. *soundness*),
- Нулто знање (енгл. *zero-knowledge*) [5].

Ове особине представљају основу функционисања ZKP система и биће детаљније описане у наставку поглавља.

2.2 Основне особине Zero-Knowledge Proof протокола

Основне особине Zero-Knowledge Proof протокола дефинишу услове које процес доказивања мора да испуни како би се обезбједила исправност провјере, спријечило прихватање нетачних тврдњи и сачувала тајност информација које доказивач посједује. То су потпуност, поузданост и нулто знање.

2.2.1 Потпуност

Потпуност (*completeness*) подразумијева да ће исправно формиран доказ за тачну тврдњу бити прихваћен од стране провјериоца са великом вјероватноћом. Другим ријечима, уколико доказивач заиста посједује потребну информацију и оба учесника правилно прате кораке дефинисане протоколом, провјерилац треба да прихвати достављени доказ [5].

Ова особина обезбјеђује да легитимни учесници система могу успјешно завршити процес доказивања. У супротном, протокол у којем би исправни докази често били одбачени не би био погодан за практичну употребу.

2.2.2 Поузданост

Поузданост (*soundness*) односи се на немогућност неовлашћеног доказивача да увјери провјериоца у нетачну тврдњу. Уколико доказивач не посједује потребну информацију, вјероватноћа да успјешно формира доказ који ће бити прихваћен треба да буде занемарљиво мала [5].

На овај начин се спречава да учесник који не познаје тајну вриједност једноставним погађањем или манипулацијом протоколом успјешно прође процес провјере.

2.2.3 Нулто знање

Нулто знање (*zero-knowledge*) представља особину по којој је ова група протокола добила назив. Она подразумијева да провјерилац током процеса доказивања не сазнаје ништа више од чињенице да је тврдња коју доказивач доказује тачна [5].

То значи да информације размјењене током протокола не омогућавају провјериоцу да реконструише тајну коју доказивач посједује. На овај начин доказивач може више пута доказивати посједовање одређене информације без потребе да је директно открије другој страни.

2.3 Начин функционисања *Zero-Knowledge Proof* протокола

Процес доказивања заснива се на јавној тврдњи и приватној информацији која се у литератури назива свједоком (енгл. *witness*). Доказивач користи свједока како би формирао податке којима потврђује истинитост тврдње. При томе, свједок се не открива провјериоцу.

Провјерилац на основу примљених података и јавно доступних параметара провјерава исправност доказа. Резултат провјере представља одлуку о прихватању или одбијању доказа. Прихватање потврђује да је тврдња исправна, без откривања приватне информације на којој се доказ заснива [5].

Конкретан ток доказивања зависи од примијењеног протокола. Према начину комуникације између доказивача и провјериоца, ZKP протоколи дијеле се на интерактивне и неинтерактивне. Њихове основне особине и разлике биће описане у наставку.

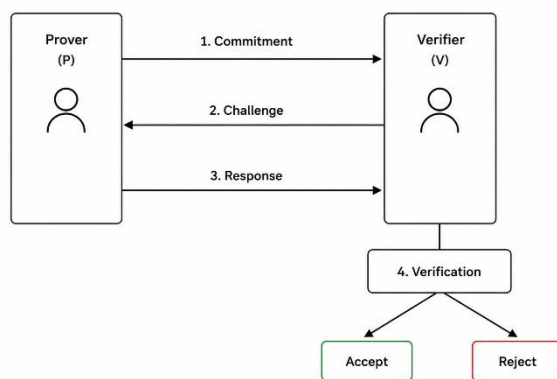
2.4 Интерактивни ZKP

Интерактивни *Zero-Knowledge Proof* протоколи подразумевају непосредну комуникацију између доказивача и провјериоца. Процес доказивања одвија се размјеном више порука [5]. Провјерилац поставља изазов, а доказивач на њега формира одговор користећи тајну информацију коју посједује. На основу добијеног одговора провјерилац прихвата или одбија доказ.

Велики број интерактивних протокола, укључујући *Schnorr* протокол, заснива се на три корака: формирању обавезе (енгл. *commitment*), задавању изазова (енгл. *challenge*) и слању одговора (енгл. *response*) [7]. Доказивач најприје генерише случајну вриједност и на основу ње формира обавезу коју просљеђује провјериоцу. Ова вриједност не треба да омогући провјериоцу да на основу ње одреди тајну коју доказивач посједује.

Након пријема обавезе, провјерилац генерише случајни изазов и просљеђује га доказивачу. Доказивач користећи примљени изазов, своју тајну и претходно кориштене податке, формира одговор. На основу добијене обавезе, послатог изазова и примљеног одговора, провјерилац провјерава да ли је испуњен услов дефинисан конкретним протоколом и доноси одлуку о прихватању или одбијању доказа.

Основни ток комуникације између доказивача и провјериоца приказује Слика 2.1.



Слика 2.1. Интеракција између доказивача и провјериоца код интерактивног ZKP протокола (ауторски приказ према [7])

Једна интеракција између доказивача и провјериоца не мора увијек бити довољна да обезбиди жељени ниво сигурности. У зависности од протокола, поступак се може поновити више пута са различитим случајно изабраним изазовима. Сваким успјешним понављањем смањује се вјероватноћа да доказивач који не посједује одговарајућу тајну успјешно увјери провјериоца у исправност своје тврдње [5].

Важну улогу у оваквим протоколима има случајност изазова који генерише провјерилац. Доказивач не треба унапријед да зна који ће изазов добити, јер би у супротном могао припремити одговор без стварног посједовања информације чије знање доказује. Изазов се генерише тек након што је доказивач формирао и послао обавезу, чиме се спречава прилагођавање почетне поруке конкретном изазову.

Један од познатих примјера интерактивног доказивања знања представља *Schnorr* протокол, који ће бити детаљније описан у Одјелку 2.6 [7].

Предност интерактивних ZKP протокола огледа се у томе што провјерилац активно учествује у процесу доказивања и генерисањем непредвидивог изазова утиче на

формирање доказа. Међутим, потреба за размјеном више порука може представљати недостатак у дистрибуираним системима, јер повећава број комуникационих корака између учесника [11].

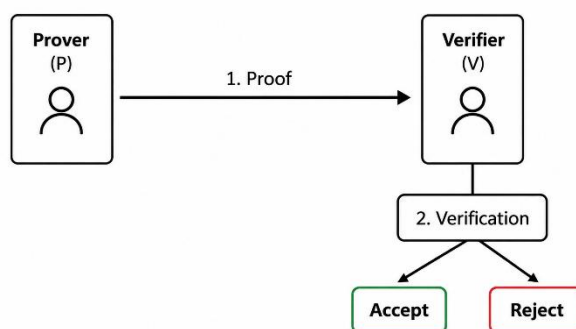
Да би се смањила потреба за оваквом интеракцијом, развијени су неинтерактивни *Zero-Knowledge Proof* протоколи код којих доказивач може формирати доказ који се касније провјерава без размјене више порука са провјериоцем [11]. Ова врста протокола биће детаљније описана у наставку.

2.5 Неинтерактивни ZKP

Неинтерактивни *Zero-Knowledge Proof* протоколи представљају протоколе код којих није потребна непосредна размјена више порука између доказивача и провјериоца током процеса доказивања. Доказивач самостално формира доказ на основу информације коју посједује и одговарајућих јавних параметара, након чега га просљеђује провјериоцу. Провјерилац затим може да провјери исправност примљеног доказа без додатне комуникације са доказивачем. На тај начин задржавају се основне особине ZKP протокола, док се истовремено смањује потреба за интеракцијом између учесника [12].

За разлику од интерактивних протокола, код којих провјерилац активно учествује у процесу генерисањем изазова, код неинтерактивног приступа доказ садржи информације потребне за његову провјеру. Након формирања, доказ се може пренијети и провјерити независно од процеса његовог генерисања, што омогућава примјену ових протокола и у системима у којима непосредна комуникација између учесника није практична.

Основни ток комуникације код неинтерактивног протокола приказан је на Слика 2.2.



Слика 2.2. Ток комуникације код неинтерактивног ZKP протокола (ауторски приказ)

Први неинтерактивни *Zero-Knowledge* системи заснивали су се на претпоставци да доказивач и провјерилац имају приступ заједничком случајном низу (енгл. *Common Random String*) генерисаном прије процеса доказивања [12]. Овај приступ представља један облик модела заједничког референтног низа (енгл. *Common Reference String*, CRS).

Други значајан приступ за уклањање интеракције представља *Fiat-Shamir* трансформација [13]. Код овог приступа случајни изазов провјериоца замјењује се вриједношћу добијеном примјеном криптографске хеш функције над обавезом и осталим подацима протокола. Доказивач на основу добијеног изазова и тајне информације формира доказ. Провјерилац самостално израчунава исти изазов и провјерава исправност примљеног доказа.

Неинтерактивни приступ смањује број комуникационих корака, јер доказивач може самостално формирати доказ, а провјерилац га накнадно провјерити. Ова особина је нарочито погодна за дистрибуиране системе, у којима може поједноставити примјену механизма доказивања између различитих компоненти система. Неинтерактивни ZKP протоколи представљају основу бројних савремених система доказивања. Конкретни системи могу се разликовати према начину генерисања јавних параметара, величини доказа, времену потребном за генерисање и провјеру, као и криптографским

претпоставкама на којима се заснива њихова безбједност. Због тога избор конкретног система зависи од захтјева окружења у којем се примјењује.

2.6 Schnorr протокол

Schnorr протокол је криптографски протокол којим доказивач доказује познавање тајне вриједности без њеног откривања провјериоцу. Његова сигурност заснива се на тежини проблема дискретног логаритма [7]. У овом раду протокол се примјењује над групом заснованом на елиптичкој кривој, због чега се користи адитивна нотација.

Доказивач посједује тајну вриједност x на основу које формира јавни кључ:

$$Y = xG,$$

при чему G представља генератор групе. Израчунавање јавног кључа Y на основу познатих вриједности x и G једноставно је, док је одређивање x на основу Y и G рачунски тешко. Ова особина представља проблем дискретног логаритма на елиптичким кривама.

На почетку доказивања доказивач бира случајну вриједност r и формира обавезу:

$$R = rG$$

Након пријема обавезе, провјерилац генерише изазов c . Доказивач затим формира одговор:

$$z = r + cx$$

Доказ се прихвата ако је испуњена једнакост:

$$zG = R + cY$$

Једнакост важи јер се замјеном $z = r + cx$ и $Y = xG$ добија:

$$zG = (r + cx)G = rG + c(xG) = R + cY$$

У основном облику *Schnorr* протокол је интерактиван. У имплементацији описаној у овом раду интеракција се уклања примјеном *Fiat-Shamir* трансформације, при чему се изазов c израчунава употребом криптографске хеш функције. Детаљи формирања изазова и примјене протокола биће представљени у поглављу о имплементацији библиотеке.

2.7 Примјене Zero-Knowledge Proof протокола

Примјена Zero-Knowledge Proofs протокола не ограничава се само на криптографске протоколе и теоријске системе. Захваљујући могућности да једна страна докаже истинитост одређене тврдње без откривања саме тајне која ту тврдњу потврђује, ZKP протоколи се могу користити у различитим областима у којима су приватност и сигурност од посебног значаја.

2.7.1 Блокчејн и криптовалуте

ZKP протоколи имају значајну примјену у *blockchain* системима, посебно у ситуацијама када је потребно потврдити валидност трансакције без откривања свих података о трансакцији. Традиционалне *blockchain* трансакције често захтијевају да одређени подаци буду јавно доступни како би други учесници могли провјерити њихову исправност. Коришћењем ZKP протокола могуће је раздвојити информацију о валидности од информације која открива саме податке.

Један од познатих примјера јесте *Zerocash*, систем који користи доказе знања како би омогућио верификацију одређених услова трансакције уз очување приватности података о трансакцији [14]. На овај начин *blockchain* технологија може задржати могућност јавне провјере, док се истовремено смањује количина информација која мора бити откривена другим учесницима система.

ZKP технологија се због тога користи и у савременим *blockchain* системима за приватност трансакција, скалабилност и доказивање одређених стања без откривања цјелокупних података.

2.7.2 Дигитални идентитет и *credentials*

Још једна значајна област примјене су системи дигиталног идентитета. У класичном систему корисник често мора открити већи број личних података како би доказао да испуњава одређени услов. На примјер, приликом доказивања пунољетности може бити потребно доставити документ који садржи и друге податке који нису релевантни за саму провјеру.

ZKP омогућава да се докаже само тражена особина. Корисник, на примјер, може доказати да је старији од одређене старосне границе без откривања тачног датума рођења.

Овакав приступ омогућава да се верификатору открије искључиво информација неопходна за доношење одлуке, без изношења вишка података [15]. Због тога су ZKP протоколи значајни за развој система дигиталног идентитета који истовремено захтијевају сигурност и очување приватности.

2.7.3 Електронско гласање

ZKP протоколи могу се примијенити и у системима електронског гласања. У таквим системима постоји потреба да се докаже да је гласање извршено према дефинисаним правилима, али без откривања избора који је гласач направио [16].

Доказ знања омогућава систему да провјери да је глас коректно конструисан и да гласач испуњава услове за учешће у изборном процесу, без увида у садржај гласа. Тиме се провјера валидности гласа потпуно раздваја од информације о самом избору.

Ова примјена илуструје кључну предност ZKP технологије: систем може верификовати да је одређена радња дозвољена и исправно извршена, без потребе за приступом свим подацима на основу којих је та радња спроведена.

За овај рад посебно је значајна примјена ZKP протокола у аутентификацији. Доказивач, умјесто слања тајне информације, доказује да је посједује. Овај приступ може се примијенити и приликом аутентификације између софтверских сервиса, што ће бити детаљније размотрено у наставку рада [6], [11].

2.8 Аутентификација сервиса у микросервисним системима

Ова цјелина биће посвећена могућности примјене *Zero-Knowledge Proof* протокола у процесу аутентификације између сервиса. Најприје ће бити размотрени неки од традиционалних приступа који се користе у ову сврху, као и безбједносни ризици који се могу јавити приликом њихове употребе. Након тога биће описан начин на који се ZKP протоколи могу примијенити као алтернативни приступ аутентификацији, са посебним освртом на разлике у начину управљања тајним подацима.

2.8.1 Традиционални приступи аутентификацији сервиса

За аутентификацију сервиса у дистрибуираним системима могу се користити различити приступи. Неки од најчешће коришћених заснивају се на употреби API кључева, приступних токена и дигиталних сертификата. Сваки од ових приступа омогућава сервису који прима захтјев да изврши одређену провјеру прије него што дозволи приступ својим функционалностима.

Један од приступа јесте употреба API кључева. API кључ представља тајну вриједност која се додјељује одређеном сервису или клијенту. Приликом слања захтјева ова вриједност се просљеђује сервису којем се приступа. Сервис затим провјерава примљени кључ и на основу тога одређује да ли ће дозволити приступ траженој функционалности [8]. Приликом употребе овог приступа битно је водити рачуна о начину на који се API кључ чува и преноси. На примјер, осјетљиви подаци не би требало да се просљеђују у URL адреси захтјева, јер на тај начин могу бити забиљежени у различитим системским логовима [8].

Други начин заснива се на употреби приступних токена. Један од формата који се користи за представљање оваквих података је *JSON Web Token* (JWT). JWT представља компактан формат за пренос скупа тврдњи (claims) између различитих страна. Ове тврдње се представљају у облику JSON објекта [9]. Токен може садржати различите информације, као што су подаци о издаваоцу, субјекту којем припада, примаоцу и времену до којег токен важи [9].

Поред API кључева и приступних токена, аутентификација између сервиса може се реализовати и помоћу дигиталних сертификата. Један од приступа који се користи у микросервисним системима је међусобна аутентификација заснована на протоколу *Transport Layer Security* (TLS), позната као *Mutual TLS* (mTLS). Код стандардне TLS комуникације клијент провјерава идентитет сервера. Код mTLS приступа провјера се врши у оба смјера, тако да се аутентификују обје стране које учествују у комуникацији [10].

Употребом mTLS приступа могуће је истовремено успоставити заштићену комуникацију и извршити међусобну аутентификацију сервиса. Овакав приступ је посебно погодан за комуникацију између микросервиса, гдје је потребно провјерити идентитет сервиса прије него што му се дозволи приступ одређеним ресурсима [10].

Сваки од описаних приступа захтијева управљање одређеним подацима који се користе у процесу аутентификације. Код API кључева потребно је заштитити сам кључ, код приступа заснованих на токенима потребно је водити рачуна о њиховом издавању и употреби, док mTLS захтијева управљање сертификатима и одговарајућим приватним кључевима. Због тога ће у наредном одјелку бити размотрени безбједносни ризици традиционалних приступа аутентификацији.

2.8.2 Безбједносни ризици традиционалних приступа

Претходно описани приступи омогућавају аутентификацију између сервиса, али њихова употреба доноси и одређене безбједносне ризике. Посебно је битно водити рачуна о подацима који се користе за доказивање идентитета сервиса, јер њихово компромитовање може омогућити неовлашћен приступ систему.

Код аутентификације помоћу API кључева главни проблем представља потреба за чувањем тајне вриједности. Уколико неовлашћени корисник дође до API кључа, може га користити за слање захтјева у име сервиса којем тај кључ припада. Због тога је потребно обезбиједити сигурно чување и пренос кључева, као и њихову замјену у случају компромитовања [8].

Сличан проблем постоји и код приступних токена. Уколико нападач дође до важећег токена, може покушати да га искористи за приступ функционалностима за које је токен намијењен. Један од начина за смањење овог ризика је ограничавање времена важења токена. На тај начин се смањује период у којем компромитовани токен може бити искориштен [9].

Код mTLS приступа потребно је на одговарајући начин управљати сертификатима и приватним кључевима сервиса. Приватни кључ мора остати заштићен, јер његово компромитовање нарушава повјерење у идентитет сервиса. Поред тога, у системима са великим бројем микросервиса потребно је водити рачуна о издавању, обнављању и повлачењу сертификата [10].

Заједничка карактеристика наведених приступа јесте потреба за управљањем осјетљивим подацима који се користе током аутентификације. ZKP протоколи не уклањају потребу за заштитом тајне вриједности, али омогућавају да се она не преноси другом сервису током провјере. На тај начин смањује се ризик од њеног откривања приликом комуникације.

2.8.3 ZKP као приступ аутентификацији сервиса

Zero-Knowledge Proof протоколи могу се примијенити и у процесу аутентификације. Основна идеја овог приступа је да сервис докаже да посједује одговарајућу тајну, а да при томе саму тајну не просљеђује другом сервису [11].

Приликом аутентификације сервис на основу тајне коју посједује формира доказ. Други сервис затим провјерава примљени доказ и на основу резултата провјере одлучује да ли ће дозволити приступ траженој функционалности. На овај начин тајна вриједност остаје позната сервису који је посједује и не открива се током процеса провјере [11].

Главна разлика у односу на приступ заснован на API кључу је у податку који се просљеђује приликом аутентификације. Код API кључа сервис другом сервису доставља вриједност која се користи за провјеру приступа. Код ZKP приступа умјесто саме тајне просљеђује се доказ формиран на основу ње. Сервис који прима захтјев на тај начин може провјерити посједовање одговарајуће тајне без потребе да му она буде откривена.

Ова особина ZKP протокола представља основу приступа који ће бити кориштен приликом развоја библиотеке описане у наредном поглављу

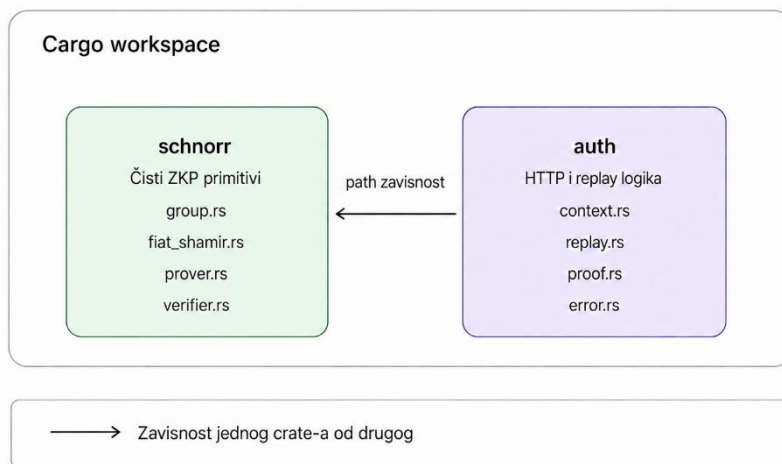
3 Пројектовање и имплементација ZKP библиотеке

У овом поглављу биће описани пројектовање и имплементација библиотеке за аутентификацију сервиса примјеном *Zero-Knowledge Proof* протокола. Најприје ће бити представљени архитектура и унутрашња организација библиотеке, као и њен јавни API. Након тога биће објашњени поступци генерисања кључева, формирања неинтерактивног *Schnorr* доказа и његове провјере. Посебна пажња биће посвећена провјери контекста захтјева и заштити од *replay* напада. На крају поглавља биће приказан начин тестирања имплементираних функционалности библиотеке.

3.1 Архитектура библиотеке

3.1.1 Унутрашња организација библиотеке

Библиотека је организована као *Cargo workspace* који садржи два *crate* пакета: *schnorr* и *auth*. Оваква организација омогућава раздвајање чисте криптографије од логике намијењене аутентификацији HTTP захтјева. Организација библиотеке и зависност између њених *crate* пакета приказане су на Слика 3.1.



Слика 3.1 Архитектура и организација библиотеке за ZKP аутентификацију

Crate schnorr садржи имплементацију *Schnorr* протокола која није повезана са HTTP комуникацијом. Његове одговорности обухватају дефинисање криптографских типова, формирање изазова, генерисање доказа и њихову провјеру. У *Schnorr crate* пакету се налазе четири модула: *group*, *fiat_shamir*, *prover* и *verifier*.

Auth crate зависи од *schnorr crate* пакета. Зависност је дефинисана у датотеци *Cargo.toml* посредством *path* атрибута. *Auth crate* додаје логику специфичну за HTTP аутентификацију између микросервиса. У *auth crate* пакету налазе се модули *context*, *replay*, *proof* и *error*.

Овакво раздвајање има више предности. *Schnorr crate* остаје независан и може се засебно тестирати и провјеравати. *Auth crate* користи криптографске типове и функције које излаже *schnorr crate*, без поновне имплементације *Schnorr* протокола. Криптографски дио библиотеке тако се може користити и независно од аутентификационог слоја и микросервисне комуникације. На тај начин, *Schnorr* протокол и ZKP механизми могу се примјењивати и у другим контекстима. Оваква организација уједно олакшава будуће проширивање библиотеке другим ZKP шемама, попут *zk-SNARK* доказа. У том случају,

нова криптографска имплементација могла би бити издвојена у засебан *crate*, док би постојећи аутентификациони слој био прилагођен њеном коришћењу.

3.1.2 Јавни API библиотеке

Библиотека је организована тако да корисник не мора директно да ради са свим унутрашњим детаљима *Schnorr* протокола. Јавни API излаже типове и функције које су потребне за генерисање кључева, формирање доказа, провјеру доказа и употребу доказа у контексту HTTP аутентификације.

Главни типови које библиотека излаже су *SecretKey*, *PublicKey*, *SchnorrProof*, *AuthContext*, *AuthProofBundle* и *ReplayProtector*. Типови *SecretKey* и *PublicKey* представљају кључеве који се користе приликом формирања и провјере доказа. *SchnorrProof* садржи формиран *Schnorr* доказ. Њихова структура и поступак генерисања биће детаљније описани у наредним одјељцима.

За аутентификацију HTTP захтјева користи се тип *AuthContext*, који садржи податке о захтјеву за који се доказ формира. Тип *AuthProofBundle* обједињује јавни кључ, контекст захтјева и *Schnorr* доказ. *ReplayProtector* служи за препознавање поновљене употребе истог доказа.

Функција *create_auth_proof* представља главну функцију коју користи сервис који формира доказ. Она као улаз добија приватни кључ и *AuthContext*, а као резултат враћа *AuthProofBundle*.

На страни сервиса који врши провјеру користи се функција *verify_auth_proof*. Она прима достављени *AuthProofBundle*, очекивани јавни кључ, податке стварног HTTP захтјева и *ReplayProtector*. На основу ових података функција извршава контекстуалну и криптографску провјеру доказа. Поступак провјере контекста и заштите од *replay* напада биће детаљније описан у одјељку 3.5.

Најважније функције јавног API интерфејса представљене су у Табела 3.1.

Табела 3.1. Најважније функције јавног API интерфејса

Функције јавног API интерфејса	Опис функције
<code>SecretKey::random</code>	Генерише приватни кључ
<code>SecretKey::public key</code>	Изводи јавни кључ из приватног
<code>prove non interactive</code>	Формира основни неинтерактивни <i>Schnorr</i> доказ
<code>verify non interactive</code>	Провјерава основни неинтерактивни <i>Schnorr</i> доказ
<code>AuthContext::new</code>	Формира контекст HTTP захтјева
<code>create_auth_proof</code>	Формира доказ за HTTP аутентификацију
<code>verify_auth_proof</code>	Провјерава доказ за HTTP аутентификацију
<code>ReplayProtector::new</code>	Креира структуру за праћење искориштених <i>nonce</i> вриједности

Овако дефинисан API раздваја криптографске детаље од употребе библиотеке у микросервисној архитектури. Сервис који шаље захтјев треба да позове функцију за формирање доказа, док сервис који прима захтјев треба да позове функцију за провјеру доказа. На тај начин библиотека може бити интегрисана у различите сервисе без потребе да сваки сервис директно имплементира *Schnorr* протокол.

3.2 Генерисање кључева

Први корак је генерисање пара кључева који ће се користити приликом формирања и провјере доказа. Пар се састоји од приватног и одговарајућег јавног кључа. Приватни кључ остаје познат само сервису којем припада, док јавни кључ мора бити доступан сервисима који провјеравају његове доказе.

За имплементацију кориштена је *Ristretto* група из библиотеке *curve25519-dalek*. Приватни кључ математички се посматра као случајно генерисани скалар, док се одговарајући јавни кључ Y изводи множењем скалара генератором групе G .

$$Y = xG$$

Сигурност овог поступка заснива се на тежини проблема дискретног логаритма. Иако је јавни кључ доступан другим сервисима, из њега није практично изводљиво израчунати одговарајући приватни кључ.

Приватни кључ генерише се функцијом *random*, чија је имплементација приказана у Листинг 3.1.

```
1 // random generation of secret key
2 pub fn random(rng: &mut (impl RngCore + CryptoRng)) -> Self {
3     Self(Scalar::random(rng).to_bytes())
4 }
```

Листинг 3.1 Имплементација функције за генерисање тајног кључа

Функција прима криптографски безбједан генератор случајних бројева и помоћу њега формира случајни скалар. Непредвидивост добијене вриједности има значајну безбједносну улогу, јер сигурност *Schnorr* протокола зависи од немогућности нападача да погоди или предвиди приватни кључ. Уколико би приватни кључ био генерисан употребом недовољно безбједног извора случајности, нападач би могао лакше да га одреди и тиме преузме идентитет сервиса приликом формирања доказа.

Одговарајући јавни кључ изводи се функцијом *public_key*, приказаном у Листинг 3.2.

```
1 // derivation of public key from secret key
2 // Y = x*G
3 pub fn public_key(&self) -> PublicKey {
4     PublicKey::from_point(&(self.scalar() * RISTRETTO_BASEPOINT_POINT))
5 }
```

Листинг 3.2 Имплементација функције за извођење јавног кључа из тајног кључа

Функција претвара приватни кључ у скалар и множи га са генератором *RISTRETTO_BASEPOINT_POINT*. Добијена тачка просљеђује се функцији *PublicKey::from_point*, која формира одговарајући јавни кључ.

У имплементацији су приватни и јавни кључ представљени низовима од 32 бајта, као што је приказано у Листингу 3.3. Јавни кључ се чува у компресованом облику, погодном за серијализацију, пренос преко мреже и каснију декомпресију приликом провјере доказа.

1	// representation of secret and public key
2	#[derive(Clone, Debug, PartialEq, Eq, Serialize, Deserialize)]
3	pub struct SecretKey([u8; 32]);
4	
5	#[derive(Clone, Debug, PartialEq, Eq, Serialize, Deserialize)]
6	pub struct PublicKey([u8; 32]);

Листинг 3.3 Дефиниција типова за представљање тајног и јавног кључа

3.3 Формирање доказа

Након генерисања пара кључева, сервис који жели да докаже свој идентитет формира *Schnorr* доказ. Доказ формира сервис који иницира интерни позив. Циљ доказа је да сервис покаже да познаје приватни кључ који одговара јавном кључу, без откривања самог приватног кључа.

Формирање доказа почиње генерисањем случајног скалара r . Ова вриједност користи се само приликом формирања једног доказа и не смије се поново употребити. Поновна употреба истог скалара r приликом формирања различитих доказа могла би угрозити приватни кључ. На основу њега формира се *commitment*:

$$R = rG$$

гдје је G генератор групе, а R *commitment* који се шаље као дио доказа. *Commitment* представља тачку групе и не открива саму вриједност r .

У имплементацији библиотеке овај корак је реализован у функцији *generate_interactive_commitment*. Најприје се генерише случајни скалар, а затим се множењем тог скалара генератором групе добија *commitment*. Генерисани скалар и *commitment* чувају се у структури *ProverState*.

Имплементација описаног поступка приказана је у Листинг 3.4.

1	// generation of an interactive commitment
2	pub fn generate_interactive_commitment(rng: &mut (impl RngCore + CryptoRng)) ->
3	ProverState {
4	let nonce = Scalar::random(rng); // r
5	let commitment = Commitment::from_point(&(nonce * generator())); // R = r*G
6	ProverState { nonce, commitment }
7	}

Листинг 3.4 Имплементација функције за генерисање интерактивне *commitment* вриједности

Случајни скалар r у имплементацији је представљен промјенљивом *nonce*, док га не треба поистовјетити са *nonce* вриједношћу HTTP захтјева која се користи за заштиту од *replay* напада.

Након формирања *commitment* вриједности потребно је добити *challenge* вриједност s . Као што је већ поменуто, у класичном интерактивном *Schnorr* протоколу *challenge* генерише провјерилац и шаље га доказивачу. Међутим, у овој библиотеци користи се неинтерактивна варијанта протокола, погоднија за HTTP комуникацију између микросервиса. *Challenge* се добија примјеном *Fiat-Shamir* трансформације, односно хеширањем јавних података доказа и контекста захтјева.

У *challenge* улазе јавни кључ, *commitment* и додатни контекст. У случају аутентификације HTTP позива, контекст обухвата идентификатор сервиса који шаље захтјев, идентификатор циљног сервиса, HTTP методу, путању, хеш тијела захтјева, *nonce* вриједност захтјева и *timestamp*. На овај начин доказ постаје везан за конкретан HTTP захтјев. Укључивање наведених података у поступак извођења изазова има значајну безбједносну улогу. Промјена било ког дијела HTTP захтјева, као што су метода, путања или тијело захтјева,

доводи до другачије *challenge* вриједности. Управо због тога претходно формирани доказ више неће бити исправан. На тај начин нападач не може преузети доказ из једног захтјева и искористити га за приступ другом ресурсу, измјену садржаја захтјева или комуникацију са другим сервисом. И доказивач и провјерилац независно изводе исту *challenge* вриједност из истих података, па њено непосредно слање није потребно.

Извођење изазова примјеном *Fiat-Shamir* трансформације реализовано је функцијом *challenge_scalar*, чија је имплементација приказана у Листинг 3.5.

```

1 // derivation of the Fiat-Shamir challenge
2 pub(crate) fn challenge_scalar(&self, public_key: &PublicKey, commitment:
3 &Commitment,) -> Scalar {
4     let mut hasher = Sha512::new();
5     hasher.update(b"zkp-auth-schnorr-fiat-shamir-v1"); // domain separation
6     hasher.update(&(self.label.len() as u64).to_le_bytes());
7     hasher.update(&self.label);
8     hasher.update(public_key.as_bytes()); // include public key Y
9     hasher.update(commitment.as_bytes()); // include commitment R
10
11     for message in &self.messages {
12         hasher.update(message); // include additional context
13     }
14
15     let digest: [u8; 64] = hasher.finalize().into(); // compute hash
16     Scalar::from_bytes_mod_order_wide(&digest) // convert hash to challenge
17     scalar c
18 }

```

Листинг 3.5. Имплементација функције за извођење изазова примјеном Fiat-Shamir трансформације

Када су познати случајни скалар r , *challenge* вриједност c и приватни кључ x , доказивач израчунава *response* вриједност z :

$$z = r + cx$$

Ова вриједност представља одговор у *Schnorr* протоколу. Иако *response* зависи од приватног кључа, из саме вриједности z није могуће израчунати приватни кључ, јер је у изразу присутан и непознати случајни скалар r . Имплементација функције за формирање *Schnorr* одговора и доказа приказана је у Листинг 3.6.

```

1 // generation of the Schnorr response and proof
2 pub fn respond(self, secret_key: &SecretKey, challenge: Scalar) -> SchnorrProof
3 {
4     // compute the response: z = r + c * x
5     let response = self.nonce + challenge * secret_key.scalar();
6     // construct the Schnorr proof
7     SchnorrProof {
8         commitment: self.commitment,
9         response: response.to_bytes(),
10     }
11 }

```

Листинг 3.6 Имплементација функције за формирање Schnorr одговора и доказа

Конечни *Schnorr* доказ састоји се од *commitment* вриједности R и *response* вриједности z . У имплементацији је овај доказ представљен структуром *SchnorrProof*, која садржи *commitment* и *response*. У неинтерактивној варијанти доказ се формира функцијом *prove_non_interactive*, која обједињује генерисање *commitment* вриједности, извођење *challenge* и израчунавање *response* вриједности. Имплементација функције за генерисање неинтерактивног *Schnorr* доказа приказана је у Листинг 3.7.

```
1 // generation of a non-interactive Schnorr proof
2 pub fn prove_non_interactive(rng: &mut (impl RngCore + CryptoRng), secret_key:
3 &SecretKey, transcript: &Transcript,
4 ) -> (PublicKey, SchnorrProof) {
5
6     // derive the public key:  $Y = x * G$ 
7     let public_key = secret_key.public_key();
8
9     // generate the commitment:  $R = r * G$ 
10    let state = generate_interactive_commitment(rng);
11
12    // derive the Fiat-Shamir challenge:  $c = H(Y, R, context)$ 
13    let challenge = transcript.challenge_scalar(&public_key,
14 state.commitment());
15
16    // generate the Schnorr response and proof:  $z = r + c * x$ 
17
18    let proof = state.respond(secret_key, challenge);
19    (public_key, proof)
20 }
```

Листинг 3.7. Имплементација функције за генерисање неинтерактивног *Schnorr* доказа

За потребе аутентификације микросервиса, основни *Schnorr* доказ се додатно пакује у структуру *AuthProofBundle*. Она садржи јавни кључ сервиса, контекст захтјева и сам *Schnorr* доказ. Ова структура се затим серијализује и шаље у HTTP заглављу интерног захтјева.

3.4 Провјера доказа

Након формирања доказа потребно је извршити провјеру његове исправности. За провјеру се користе јавни кључ сервиса, примљени *Schnorr* доказ и исти контекст који је кориштен приликом формирања доказа.

Основна провјера *Schnorr* доказа заснива се на једначини:

$$zG = R + cY$$

гдје је z *response* вриједност, G генератор групе, R *commitment*, c *challenge* вриједност, а Y јавни кључ сервиса који доказује идентитет. Ако ова једначина важи, провјерилац може закључити да је доказивач у тренутку формирања доказа познавао приватни кључ x који одговара јавном кључу:

$$Y = xG$$

У имплементацији библиотеке ова провјера је реализована у функцији *verify_interactive*, чија је имплементација приказана у Листинг 3.8.

```
1 pub fn verify_interactive(public_key: &PublicKey, commitment: &Commitment,
2 challenge: Scalar, response: [u8; 32],) -> Result<(), VerificationError> {
3
4     // Decode the public key Y into a curve point
5     let public_key_point = public_key.point()
6         .ok_or(VerificationError::InvalidPublicKey)?;
7
8     // Decode the commitment R
9     let commitment_point = commitment.point()
10        .ok_or(VerificationError::InvalidCommitment)?;
11
12    // Parse z from bytes into a Scalar
13    let response_scalar =
14    Option::<Scalar>::from(Scalar::from_canonical_bytes(response))
15        .ok_or(VerificationError::InvalidResponse)?;
16
17    // Left-hand side of the equation: z * G
18    let left = response_scalar * generator();
19
20    // Right-hand side of the equation: R + c * Y
21    let right = commitment_point + challenge * public_key_point;
22
23    if left == right{
24        Ok(())
25    }
26    else {
27        Err(VerificationError::InvalidProof)
28    }
29 }
```

Листинг 3.8 Имплементација функције за провјеру интерактивног *Schnorr* доказа

Најприје се јавни кључ и *commitment* декомпресују у тачке *Ristretto* групе, а *response* се претвара у скалар. Уколико неки од ових података није исправно представљен, верификација се прекида грешком.

У имплементацији се за аутентификацију микросервиса користи неинтерактивна варијанта *Schnorr* протокола. Због тога провјерилац не добија *challenge* одвојеном поруком, већ је

поново израчунава на основу *transcript* података. *Transcript* обухвата јавни кључ, *commitment* и контекст захтјева. Ако су подаци исти као приликом формирања доказа, добија се иста *challenge* вриједност, након чега се позива основна провјера *Schnorr* једначине.

3.5 Провјера контекста захтјева и заштита од *replay* напада

Као што је описано приликом формирања доказа, подаци HTTP захтјева укључују се у Fiat-Shamir transcript и тиме утичу на изведену *challenge* вриједност. Међутим, приликом пријема захтјева није довољно само извршити математичку провјеру *Schnorr* доказа. Потребно је утврдити и да ли контекст достављен у структури *AuthProofBundle* одговара HTTP захтјеву који се тренутно обрађује, као и да ли доказ задовољава услове временске ваљаности и једнократне употребе.

Провјера комплетног аутентификационог доказа реализована је функцијом *verify_auth_proof*, приказаном у Листинг 3.9.

```
1 pub fn verify_auth_proof(bundle: &AuthProofBundle,
2     expected_public_key: &PublicKey,
3     expected_audience: &str,
4     method: &str,
5     path: &str,
6     body: &[u8],
7     now_unix_secs: u64,
8     replay_protector: &mut ReplayProtector) -> Result<(),
9     AuthVerificationError> {
10     if &bundle.public_key != expected_public_key {
11         return Err(AuthVerificationError::PublicKeyMismatch);
12     }
13
14     if bundle.context.audience != expected_audience
15         || bundle.context.method != method.to_uppercase()
16         || bundle.context.path != path
17         || bundle.context.body_sha256 != sha256_hex(body)
18     {
19         return Err(AuthVerificationError::RequestContextMismatch);
20     }
21
22     replay_protector.verify_and_store(&bundle.context, now_unix_secs)?;
23     let transcript = bundle.context.transcript();
24     verify_non_interactive(&bundle.public_key, &bundle.proof, &transcript)?;
25
26     Ok(())
27 }
```

Листинг 3.9 Провјера аутентификационог доказа

Функција, поред примљене структуре *AuthProofBundle*, добија очекивани јавни кључ, идентификатор циљног сервиса, HTTP методу, путању, стварно примљено тијело захтјева, тренутно вријеме и структуру *ReplayProtector*. На тај начин провјерилац не користи искључиво податке које је доставио пошilhaлац, већ их пореди са вриједностима које одговарају захтјеву који се тренутно обрађује.

На почетку провјере упоређује се јавни кључ из примљеног доказа са јавним кључем који је регистрован за сервис чији се идентитет провјерава. Чињеница да је доказ математички исправан не би била довољна ако би нападач могао доставити сопствени јавни кључ и доказ формиран одговарајућим приватним кључем. Зато јавни кључ мора бити познат сервису који врши провјеру и повезан са идентитетом очекиваног пошilhaоца.

Након провјере јавног кључа пореде се подаци из контекста са подацима примљеног HTTP захтјева. Циљни сервис мора одговорати сервису који обрађује захтјев, док HTTP метода и

путања морају бити једнаке вриједностима наведеним приликом формирања доказа. Такође се поново израчунава хеш примљеног тијела и пореди са хешом који је укључен у контекст. Уколико је тијело захтјева промијењено након формирања доказа, добијене вриједности се неће подударати и захтјев ће бити одбијен.

Поред везивања доказа за садржај захтјева, потребно је спријечити поновну употребу потпуно исправног захтјева. Ова врста напада назива се *replay* напад и подразумијева пресретање валидне поруке и њено поновно слање са циљем да се иста операција изврши више пута. Такав напад је посебно опасан код операција као што су резервација производа или обрада плаћања.

Заштита од *replay* напада реализована је комбинацијом *timestamp* и *nonce* вриједности. *Timestamp* означава вријеме формирања доказа и омогућава ограничавање периода у којем се доказ сматра важећим. Приликом провјере израчунава се разлика између тренутног времена и времена наведеног у контексту. Ако је дозвољени временски период прекорачен, доказ се одбија без обзира на то што може бити криптографски исправан.

Nonce представља једнократну вриједност која се генерише за сваки нови захтјев. Ако сервис прими други захтјев са истом *nonce* вриједношћу, препознаје га као покушај поновне употребе доказа и одбија његово извршавање. Искориштене вриједности потребно је чувати током периода важења доказа, након чега се могу уклонити из складишта.

У имплементацији библиотеке овај механизам реализован је типом *ReplayProtector*. Он чува искориштене *nonce* вриједности у меморији, што је довољно за потребе демонстрационе апликације. Међутим, у продукционом окружењу са више инстанци истог сервиса било би неопходно користити дијељено складиште, као што су *Redis* или база података, како би све инстанце имале приступ истом скупу искориштених *nonce* вриједности.

Након провјере јавног кључа, контекста, *timestamp* и *nonce* вриједности реконструише се *Fiat-Shamir transcript*. У њега се укључују исти подаци који су кориштени приликом формирања доказа, након чега се поново израчунава *challenge* и извршава математичка провјера *Schnorr* једначине. Доказ се прихвата само ако су успјешно завршене и контекстуалне и криптографске провјере.

Описаним приступом обезбјеђује се да доказ није само потврда познавања приватног кључа, већ и потврда да је формиран за одређени HTTP захтјев, одређени циљни сервис и ограничени временски период. На тај начин библиотека повезује *Schnorr* протокол са практичним захтјевима аутентификације у микросервисном окружењу.

3.6 Тестирање библиотеке

Након имплементације *Schnorr* протокола и механизма за аутентификацију HTTP захтјева, извршено је тестирање библиотеке. Циљ тестирања је био да се провјери исправност основног *Schnorr* доказа, као и додатних механизма који су уведени због употребе библиотеке у микросервисној архитектури.

3.6.1 Тестови у оквиру *Schnorr* crate

Тестови имплементирани у оквиру *schnorr* crate пакета покрећу се командом:

```
cargo test -p schnorr
```

Најприје су тестиране основне функционалности *schnorr* crate пакета. Првим тестом провјерава се да исправно формиран неинтерактивни *Schnorr* доказ успјешно пролази верификацију када се користе одговарајући јавни кључ и *transcript*. Имплементација овог теста приказана је на Листинг 3.10.

1	<code>#[test]</code>
2	<code>fn valid_non_interactive_proof_verifies() {</code>
3	<code> let mut rng = OsRng;</code>
4	<code> let secret = SecretKey::random(&mut rng);</code>
5	<code> let transcript = Transcript::new(b"service-auth-test");</code>
6	<code> let (public_key, proof) = prove_non_interactive(&mut rng, &secret,</code>
7	<code>&transcript);</code>
8	<code> assert!(verify_non_interactive(&public_key, &proof, &transcript).is_ok());</code>
9	<code>}</code>
10	

Листинг 3.10 Провјера исправног неинтерактивног *Schnorr* доказа

Други тест представља негативан сценарио и провјерава да доказ не може бити верификован јавним кључем који не одговара приватном кључу коришћеном за његово генерисање. Имплементација овог теста приказана је на Листинг 3.11.

1	<code>#[test]</code>
2	<code>fn proof_does_not_verify_for_different_public_key() {</code>
3	<code> let mut rng = OsRng;</code>
4	<code> let secret = SecretKey::random(&mut rng);</code>
5	<code> let other_secret = SecretKey::random(&mut rng);</code>
6	<code> let transcript = Transcript::new(b"service-auth-test");</code>
7	<code> let (_, proof) = prove_non_interactive(&mut rng, &secret, &transcript);</code>
8	<code> assert_eq!(</code>
9	<code> verify_non_interactive(&other_secret.public_key(), &proof, &transcript),</code>
10	<code> Err(VerificationError::InvalidProof)</code>
11	<code>);</code>
12	<code>}</code>

Листинг 3.11 Провјера одбијања доказа за други јавни кључ

Успјешним извршавањем наведених тестова потврђено је да имплементација прихвата исправно формиран доказ, док се покушај верификације помоћу неодговарајућег јавног кључа одбацује.

3.6.2 Тестови у оквиру *Auth crate*

Тестови имплементирани у оквиру *auth crate* пакета и покрећу се командом:

```
cargo test -p auth
```

Поред тестова основних функционалности *Schnorr* протокола, имплементирани су и тестови аутентификационог слоја. Њима се провјерава везивање доказа за контекст конкретног HTTP захтјева, заштита од поновљеног слања истог захтјева и одбијање доказа чији *timestamp* излази из дозвољеног временског интервала.

Првим тестом провјерава се да исправно формиран доказ успјешно пролази верификацију за оригиналну путању захтјева, док се употреба истог доказа са промијењеном путањом одбија уз грешку *RequestContextMismatch*. На овај начин потврђује се да је доказ везан за конкретан HTTP захтјев.

Другим тестом исти доказ се верификује два пута. Прва верификација је успјешна, након чега се попсе чува у оквиру компоненте *ReplayProtector*. Приликом поновљене провјере исти попсе је већ регистрован, због чега се захтјев одбија уз грешку *ReplayDetected*.

Трећим тестом провјерава се временска валидност доказа. Вријеме верификације постављено је изван дозвољеног временског интервала у односу на *timestamp* захтјева, па се очекује грешка *ExpiredTimestamp*.

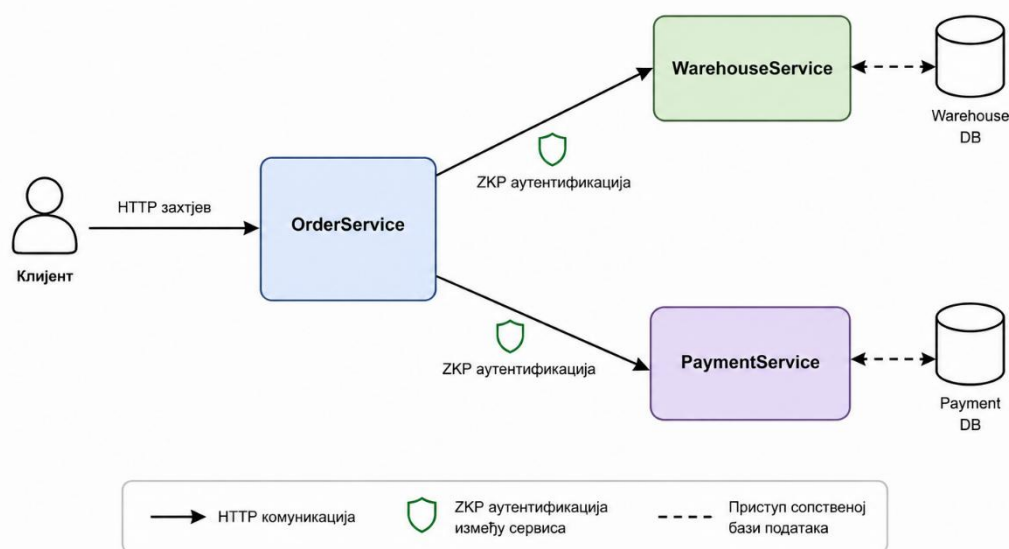
Због обима и сличности структуре са претходно приказаним тестовима, њихова комплетна имплементација није приказана у раду. Изворни код свих тестова доступан је у репозиторијуму библиотеке. Успјешним извршавањем тестова потврђено је да аутентификациони слој правилно повезује доказ са HTTP захтјевом, препознаје поновљено коришћење попсе вриједности и одбацује временски неважеће доказе.

4 Примјена библиотеке у микросервисном окружењу

За потребе приказа практичне примјене развијене библиотеке имплементирана је демонстрациона микросервисна апликација састављена од три сервиса: *OrderService*, *WarehouseService* и *PaymentService*. Апликација моделује процес креирања наруџбе, при чему се развијени механизам користи за аутентификацију интерних позива између сервиса.

4.1 Архитектура демонстрационе апликације

Сваки сервис представља независну цјелину са јасно дефинисаном одговорношћу. *OrderService* има централну улогу и оркестрира процес наручивања, док су *WarehouseService* и *PaymentService* задужени за резервацију залиха, односно обраду плаћања. Сервиси међусобно комуницирају путем *Hypertext Transfer Protocol* (HTTP) захтјева. Архитектура демонстрационе микросервисне апликације приказана је на Слика 4.1.



Слика 4.1. Архитектура демонстрационе апликације

Клијент директно приступа само *OrderService* сервису. Интерни захтјеви које овај сервис упућује сервисима *WarehouseService* и *PaymentService* садрже ZKP аутентификациони доказ. Сервис који прима захтјев провјерава доказ прије извршавања пословне операције, чиме се аутентификација реализује као обавезан корак интерне комуникације.

4.2 Имплементација демонстрационе апликације

Сва три сервиса у демонстрационој апликацији имплементирана су у програмском језику *Rust*. За организацију пројекта кориштен је *Cargo workspace*, који омогућава

организовање више међусобно повезаних *crate* пакета у оквиру једног пројекта. Сваки микросервис реализован је као засебан *crate*, чиме је задржана логичка раздвојеност сервиса и омогућено њихово независно покретање.

Радни простор организован је кроз *crate* пакете *order-service*, *warehouse-service* и *payment-service*. За дијељене компоненте издвојен је посебан *common crate*. У њему су дефинисани модели и функционалности које користи више сервиса, чиме се избјегава дуплирање заједничког кода.

За реализацију HTTP слоја коришћен је програмски оквир *Axum*, док је асинхронно извршавање операција омогућено употребом *Tokio* окружења. У оквиру сваког сервиса дефинисане су HTTP руте и одговарајуће функције за обраду пристиглих захтјева. За слање захтјева између сервиса користи се HTTP клијент, при чему се у овај дио комуникације интегрише развијена библиотека за ZKP аутентификацију.

4.2.1 Имплементација *OrderService* сервиса

OrderService представља једини сервис којем клијент директно приступа и задужен је за управљање цјелокупним током наручивања. Његова основна улога је оркестрација процеса креирања нарудбе, што подразумемијева пријем и провјеру клијентског захтјева, позивање осталих сервиса у систему и формирање јединственог одговора који се враћа клијенту. Интерну комуникацију са сервисима *WarehouseService* и *PaymentService* остварује слањем HTTP захтјева посредством модула *warehouse_client* и *payment_client*, који се налазе у оквиру пакета *client*.

У оквиру *OrderService* сервиса дефинисане су двије основне HTTP руте. Рута *GET /health* служи за провјеру доступности сервиса и враћа једноставан одговор са називом сервиса и његовим статусом. Исти модел одговора користи се за *health* руту у свим сервисима демонстрационе апликације, а приказан је у Листинг 4.1.

1	<code>#[derive(Debug, Clone, Serialize)]</code>
2	<code>pub struct HealthResponse {</code>
3	<code> pub service: &'static str,</code>
4	<code> pub status: &'static str,</code>
5	<code>}</code>

Листинг 4.1. Модел *HealthResponse*

Главна функционална рута је *POST /orders*, преко које клијент шаље захтјев за креирање нове нарудбе. Након пријема захтјева, ова рута покреће цјелокупан ток обраде нарудбе, који обухвата провјеру производа, резервисање залиха у *WarehouseService* сервису, креирање плаћања у *PaymentService* сервису и формирање коначног одговора клијенту.

Захтјев и одговор руковаоца за креирање нарудбе дефинисани су у заједничком *common crate* пакету, чиме сва три сервиса дијеле исти уговор о облику размјењиваних података. Модел захтјева обухвата податке које клијент шаље приликом наручивања, док модел одговора садржи резултат обраде нарудбе, укључујући идентификатор нарудбе, статус, идентификатор резервације, идентификатор плаћања и укупан износ. Основни модели за наручивање приказани су у Листинг 4.2.

```

1  #[derive(Debug, Clone, Deserialize)]
2  pub struct CreateOrderRequest {
3      pub customer_id: String,
4      pub product_id: String,
5      pub quantity: u32,
6      pub payment_method: String,
7  }
8
9  #[derive(Debug, Clone, Serialize)]
10 pub struct CreateOrderResponse {
11     pub order_id: Uuid,
12     pub status: OrderStatus,
13     pub reservation_id: Uuid,
14     pub payment_id: Uuid,
15     pub total_amount: u64,
16 }

```

Листинг 4.2. Основни модели коришћени за наручивање

4.2.2 Имплементација *WarehouseService* сервиса

WarehouseService је сервис задужен за управљање залихама производа и резервисање тражене количине производа у оквиру процеса креирања наруџбе. Клијент не приступа директно овом сервису, већ га позива *OrderService* путем интерног HTTP захтјева. На тај начин је логика рада са залихама издвојена у засебан микросервис.

У оквиру *WarehouseService* сервиса дефинисане су двије HTTP руте. Прва је рута *GET /health*, која је већ претходно поменута. Главна функционална рута је *POST /reservations*, преко које *OrderService* шаље захтјев за резервисање производа. Захтјев садржи идентификатор наруџбе, идентификатор производа и количину коју је потребно резервисати.

Приликом обраде захтјева за резервацију, *WarehouseService* најприје провјерава ZKP аутентификациони доказ који се налази у HTTP заглављу захтјева. Тек након успјешне провјере доказа сервис приступа пословној логици. Провјерава се да ли тражени производ постоји у складишту, да ли је тражена количина већа од нуле и да ли је доступна довољна количина производа. Уколико су услови испуњени, доступна количина се умањује за резервисану количину, а сервис враћа одговор који садржи идентификатор резервације, идентификатор производа, резервисану количину и преосталу количину на стању.

4.2.3 Имплементација *PaymentService* сервиса

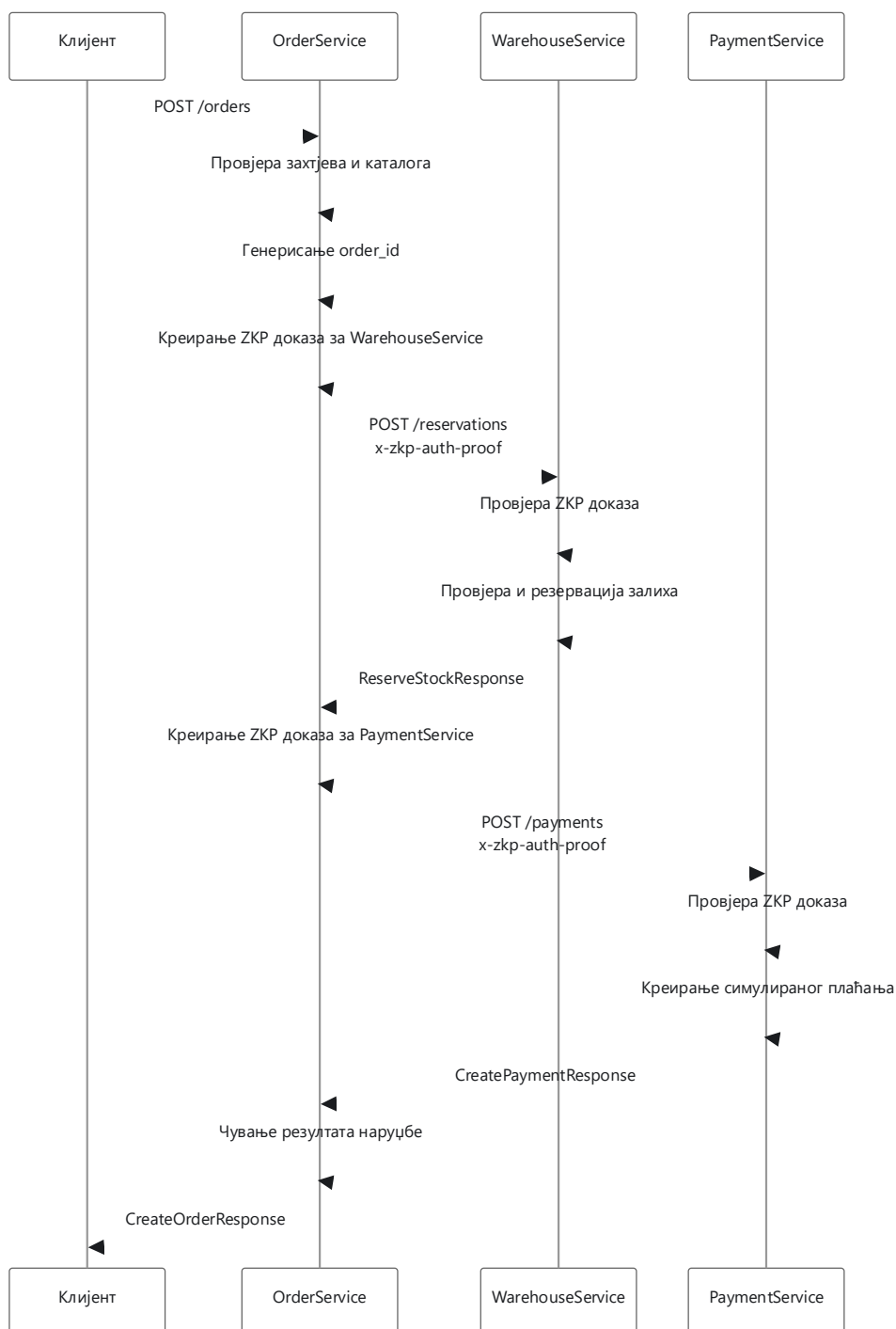
PaymentService је сервис задужен за обраду плаћања у оквиру демонстрационе микросервисне апликације. Као и *WarehouseService*, ни овај сервис није директно доступан клијенту у главном току апликације, већ га позива *OrderService* након успјешне провјере наруџбе и резервисања производа. Његова улога је да издвоји логику плаћања у посебан микросервис са јасно дефинисаном одговорношћу.

У оквиру *PaymentService* сервиса дефинисане су двије HTTP руте. Поред руте *GET /health*, главна функционална рута је *POST /payments*, преко које *OrderService* шаље захтјев за креирање плаћања. Захтјев садржи идентификатор наруџбе, укупан износ и начин плаћања.

Приликом пријема захтјева за плаћање, *PaymentService* најприје провјерава ZKP аутентификациони доказ из HTTP заглавља. На тај начин се осигурава да захтјев потиче од *OrderService* сервиса и да није измијењен током преноса. Након успјешне провјере доказа, сервис валидира податке о плаћању и креира одговор. У демонстрационој апликацији плаћање је симулирано, па сервис не комуницира са стварним платним системом, већ генерише идентификатор плаћања и враћа статус *approved*. Одговор садржи идентификатор плаћања, идентификатор наруџбе, статус плаћања и износ.

4.3 Ток рада апликације

Процес започиње слањем захтјева сервису *OrderService*, који позива *WarehouseService* ради резервације производа, а затим *PaymentService* ради обраде плаћања. Сваки интерни захтјев садржи посебан ZKP доказ који се провјерава прије извршавања операције. Ток обраде наруџбе приказан је на Слика 4.2.



Слика 4.2 Ток обраде наруџбе у демонстрационој апликацији

4.4 Интеграција ZKP библиотеке

Након имплементације основне пословне логике сервиса, развијена ZKP библиотека интегрисана је у дио апликације који реализује интерну комуникацију између микросервиса. Циљ интеграције је да сервис који шаље захтјев не просљеђује класичну тајну, као што су лозинка, API кључ или токен, већ да приложи криптографски доказ о посједовању приватног кључа. У демонстрационој апликацији ту улогу има *OrderService*, који приликом позива *WarehouseService* и *PaymentService* сервиса формира ZKP аутентификациони доказ. Сервиси који примају интерне захтјеве затим провјеравају приложени доказ на основу јавног кључа *OrderService* сервиса.

Интеграција библиотеке реализована је у два корака. Први корак је креирање доказа на страни сервиса који шаље захтјев, док је други корак провјера доказа на страни сервиса који захтјев прима. На овај начин ZKP механизам није издвојен као изолован примјер, већ је укључен у стварни ток комуникације између микросервиса.

На страни сервиса који иницира комуникацију, доказ се формира прије слања сваког интерног HTTP захтјева. За то је задужена функција `build_zkp_auth_header`, приказана у Листинг 4.3.

```
1 pub fn build_zkp_auth_header<T: Serialize>(
2     state: &AppState,
3     target_service: &str,
4     method: &str,
5     path: &str,
6     body: &T,
7 ) -> Result<String, ServiceError> {
8     let body = serde_json::to_vec(body)
9         .map_err(|error| ServiceError::internal(format!("failed to serialize
10 request: {error}")))?;
11     let nonce = Uuid::new_v4().as_bytes().to_vec();
12
13     let context = AuthContext::new(
14         ORDER_SERVICE_ID,
15         target_service,
16         method,
17         path,
18         &body,
19         current_unix_timestamp(),
20         nonce,
21     );
22
23     let mut rng = OsRng;
24     let proof = create_auth_proof(&mut rng, &state.service_secret_key,
25 context);
26     encode_auth_proof(&proof)
27         .map_err(|error| ServiceError::internal(format!("failed to encode auth
28 proof: {error}")))
29 }
```

Листинг 4.3 Формирање ZKP аутентификационог заглавља

Функција најприје серијализује тијело захтјева и формира *AuthContext*, који обухвата пошиљаоца, одредишни сервис, HTTP методу, путању, тијело захтјева, временску ознаку и *nonce*. На основу тог контекста *OrderService* позива функцију `create_auth_proof` из развијене ZKP библиотеке и креира доказ употребом свог приватног кључа. Доказ се затим

серијализује функцијом *encode_auth_proof* и додаје у HTTP заглавље *x-zkp-auth-proof*, преко којег се преноси ка другом сервису.

Потребно је нагласити да демонстрациона апликација не имплементира комплетан механизам управљања кључевима који би био неопходан у продукционом окружењу. Ради једноставнијег локалног покретања, приватни кључ *OrderService* сервиса унапријед је дефинисан у коду, односно детерминистички се изводи из фиксног текстуалног материјала. То омогућава да се при сваком покретању добије исти пар кључева и да *WarehouseService* и *PaymentService* могу одмах провјеравати доказе без додатне конфигурације.

Овакав приступ погодан је за демонстрацију интеграције библиотеке, али није безбједан за продукциону употребу, јер приватни кључ није чуван као стварна тајна. У реалном систему приватни кључ би био генерисан употребом развијене библиотеке, сачуван у безбједном складишту тајни и доступан само сервису који креира доказе. Остали сервиси би имали приступ искључиво одговарајућем јавном кључу, који би користили за провјеру примљених доказа.

На страни сервиса који прима захтјев, доказ се издваја из HTTP заглавља *x-zkp-auth-proof* и провјерава позивом функције *verify_auth_proof* из развијене библиотеке. Провјера се врши на основу јавног кључа *OrderService* сервиса и истог контекста захтјева: одредишног сервиса, HTTP методе, путање, тијела захтјева и тренутног времена. Ако доказ не одговара примљеном захтјеву или је већ искоришћен исти *nonce*, захтјев се одбија као неауторизован. Тек након успјешне провјере сервис наставља са извршавањем своје пословне логике.

Иако је библиотека имплементирана у програмском језику Rust, може се користити и у системима чији су сервиси написани у другим програмским језицима. Једна могућност је издвајање функционалности библиотеке у засебан сервис који би осталим сервисима пружао функције за генерисање и провјеру доказа путем API. Друга могућност је коришћење FFI механизма за позивање функционалности библиотеке из других програмских језика. Такође, за појединачне језике могу се имплементирати библиотеке које прате исту спецификацију протокола. У том случају потребно је обезбиједити компатибилност криптографских параметара и начина формирања порука и доказа.

4.5 Перформансе ZKP аутентификације

Поред функционалне провјере библиотеке, извршено је и мјерење времена потребног за генерисање и провјеру доказа. Циљ ових мјерења је да се процијени рачунски трошак генерисања и провјере доказа који се користе приликом ZKP аутентификације.

За мјерење времена потребног за генерисање и провјеру доказа кориштен је алат *Criterion*. Сва мјерења перформанси извршена су на рачунару са Intel Core i7 процесором и 16 GB RAM меморије, под оперативним системом Windows 11 Pro. Мјерења су извршена у оптимизованом (*release*) режиму, при чему је за сваку операцију прикупљено 200 узорака. Просјечно процијењено вријеме генерисања доказа износило је 71,223 μ s, док је за његову провјеру било потребно 81,472 μ s.

За генерисање доказа добијен је интервал од 69,799 μ s до 72,798 μ s, док се вријеме провјере кретало у интервалу од 80,772 μ s до 82,253 μ s. Добијени резултати показују да се обје операције у посматраном окружењу извршавају за мање од 0,1 ms.

Измјерене вриједности времена генерисања и провјере доказа приказане су у Табела 4.1.

Табела 4.1. Вријеме генерисања и провјере доказа

Операција	Доња граница	Процијењено вријеме	Горња граница
Генерисање доказа	69,799 μ s	71,223 μ s	72,798 μ s
Провјера доказа	80,772 μ s	81,472 μ s	82,253 μ s

Поред времена потребног за генерисање и провјеру доказа, извршено је и мјерење величине података који се користе приликом аутентификације. Мјерење је извршено над 1000 генерисаних доказа, при чему су посматране величина самог *Schnorr* доказа и величина комплетног *AuthProofBundle* објекта.

Сирови *Schnorr* доказ састоји се од 32-бајтне *commitment* вриједности и 32-бајтног одговора, због чега његова величина износи 64 В. Приликом JSON серијализације величина доказа је варијала од 240 В до 268 В, док је просјечна величина износила 255,93 В. До варијације долази због текстуалног представљања појединачних бајтова у JSON формату. Комплетан *AuthProofBundle*, који поред доказа садржи јавни кључ и контекст HTTP захтјева, имао је просјечну JSON величину од 660,93 В, са вриједностима у распону од 645 В до 673 В.

Поред JSON формата, извршено је и мјерење коришћењем бинарне серијализације. У овом случају величина *Schnorr* доказа износила је 64 В, док је величина комплетног *AuthProofBundle* објекта износила 290 В. Поређењем резултата може се уочити да бинарна серијализација производи компактнији запис у односу на JSON серијализацију. У Табела 4.2. приказане су величина доказа и аутентификационих података.

Табела 4.2 Величина доказа и аутентификационих података

Начин представљања	Минимум	Просјек	Максимум
Сирови Schnorr доказ	64 В	64 В	64 В
JSON Schnorr доказ	240 В	255,93 В	268 В
JSON AuthProofBundle	645 В	660,93 В	673 В
Bincode Schnorr доказ	64 В	64 В	64 В
Bincode AuthProofBundle	290 В	290 В	290 В

Претходно поменути тестови налазе се у директоријумима *benches* и *tests* у оквиру *auth crate* пакета. У директоријуму *benches* налазе се тестови за мјерење времена генерисања и провјере доказа, док су у директоријуму *tests* смјештени функционални тестови описани у поглављу 3.6, као и тест за мјерење величине података.

4.6 Поређење перформанси ZKP и традиционалних приступа аутентификацији

Иако су вријеме генерисања и провјере ZKP доказа већ измјерени у претходном одјелу, за потребе поређења ове операције су поново измјерене у оквиру истог *benchmark*-а као и остали приступи аутентификацији. На тај начин добијене су вриједности које су директно упоредиве са мјерењима за API кључ, JWT приступни токен и mTLS протокол. Циљ овог поређења није мјерење укупног времена аутентификације, већ поређење времена извршавања одабраних репрезентативних операција карактеристичних за сваки од наведених приступа.

За API кључ мјери се *constant-time* поређење презентованог кључа са сачуваним кључем, односно операција коју сервис прималац извршава приликом провјере. За JWT мјери се издавање и провјера токена алгоритмом HS256, са тврдњама (*claims*) описаним у поглављу 2 - издавалац, субјекат, прималац и вријеме важења. За mTLS мјери се криптографски дио узајамно аутентификованог P-256 *handshake*-а, који обухвата ECDHE размјену кључа и операције потписивања и провјере за обје стране. Мјерења обухватају само наведене операције, без мрежног преноса и осталих дијелова комуникационог протокола. Измјерене вриједности приказане су у Табела 4.3.

Табела 4.3 Вријеме извршавања репрезентативних операција - ZKP, API кључ, JWT (HS256) и mTLS (P-256)

Механизам и операција	Доња граница	Процијењено вријеме	Горња граница
ZKP - генерисање Schnorr доказа	63,826 μ s	64,294 μ s	64,804 μ s
ZKP - провјера Schnorr доказа	66,955 μ s	67,781 μ s	68,743 μ s
API кључ - <i>constant-time</i> поређење	156,07 ns	157,82 ns	159,79 ns
JWT - издавање HS256 токена	1,0828 μ s	1,0968 μ s	1,1124 μ s
JWT - провјера HS256 токена	2,6522 μ s	2,6815 μ s	2,7151 μ s
mTLS - P-256 ECDHE размјена кључа	151,26 μ s	153,17 μ s	155,41 μ s
mTLS - потписивање и провјера за обје стране	409,13 μ s	415,25 μ s	422,12 μ s

Добијени резултати показују јасну разлику у времену извршавања посматраних операција. Најмањи трошак има провјера API кључа, која у просјеку траје око 158 ns. Код JWT приступа, просјечно вријеме издавања токена износи 1,0968 μ s, док је за његову провјеру потребно 2,6815 μ s. Генерисање и провјера ZKP доказа захтијевају више времена, са просјечним вриједностима од 64,294 μ s и 67,781 μ s. Код mTLS приступа измјерене асиметричне операције имају највећи рачунски трошак. ECDHE размјена кључа траје просјечно 153,17 μ s, док за потписивање и провјеру за обје стране треба просјечно 415,25 μ s.

Ови резултати показују да је рачунски трошак ZKP аутентификације већи од трошка API кључа и JWT приступа, што је очекивано с обзиром на то да ZKP користи операције над елиптичком кривом. Са друге стране, измјерене ZKP операције имају мањи трошак од посматраних асиметричних операција mTLS протокола. При томе, ово поређење не представља директно поређење укупног времена аутентификације. На примјер, mTLS

handshake се извршава приликом успостављања TLS сесије, након чега се његов трошак може распоредити на више захтјева преко дуготрајне конекције. Насупрот томе, у развијеној библиотеци ZKP доказ се генерише и провјерава за сваки појединачни HTTP захтјев.

Поред извршног времена, наведени приступи разликују се и по начину обезбјеђивања аутентификације. API кључ и JWT са алгоритмом HS256 користе дијељену тајну, док ZKP приступ користи асиметричну криптографију, при чему сервис који врши провјеру посједује само јавни кључ. mTLS такође користи асиметричну криптографију, али аутентификацију остварује на нивоу TLS сесије. Развијена библиотека, са друге стране, повезује доказ са конкретним HTTP захтјевом кроз контекст који садржи податке као што су метод, путања, тијело захтјева, прималац и вријеме слања.

На основу добијених резултата може се закључити да ZKP приступ има већи рачунски трошак од једноставнијих механизма као што су API кључ и JWT, али да измјерено вријеме генерисања и провјере доказа остаје у реду величине десетина микросекунди. Његова практична примјена зато не зависи само од времена извршавања, већ и од захтјева система у погледу начина аутентификације, управљања тајнама и повезивања доказа са конкретним захтјевима.

5 Закључак

У овом раду представљена је имплементација библиотеке за аутентификацију између микросервиса засновану на *Zero-Knowledge Proof* приступу. Као основа библиотеке коришћен је *Schnorr* протокол, који омогућава да један сервис докаже посједовање приватног кључа без његовог откривања. Библиотека је организована као *Rust workspace* са одвојеним компонентама за основни *Schnorr* протокол и аутентификациони слој прилагођен HTTP комуникацији. Поред формирања и провјере доказа, имплементирано је везивање доказа за конкретан HTTP захтјев, као и заштита од поновљеног слања истог доказа употребом *nonce* вриједности и временске ознаке.

Практична примјена библиотеке приказана је кроз демонстрациону микросервисну апликацију састављену од сервиса *OrderService*, *WarehouseService* и *PaymentService*. У том систему клијент комуницира само са централним сервисом за наруџбе, док се интерни позиви ка сервисима за складиште и плаћање аутентификују помоћу ZKP доказа. Главни допринос рада огледа се у повезивању теоријског модела *Schnorr* доказа са практичним сценаријем сервис-сервис аутентификације, као и у јасном раздвајању криптографске логике од пословне логике микросервиса.

Са аспекта примјене у реалним системима, представљени приступ треба посматрати као алтернативу, а не као директну замјену за постојећа рјешења. ZKP омогућава доказивање посједовања приватног кључа без његовог откривања, али истовремено уводи додатну криптографску сложеност и трошак израде и провјере доказа. За већину савремених микросервисних система, mTLS и слични механизми остају практичнији због једноставније интеграције и зреле инфраструктурне подршке. Имплементирана библиотека се зато прије може посматрати као доказ изводљивости ZKP приступа и основа за сценарије у којима су његова специфична безбједносна својства од посебног значаја.

Будући правци развоја могу обухватити унапређење начина управљања јавним кључевима сервиса, на примјер кроз регистар сервиса или интеграцију са постојећом инфраструктуром за управљање идентитетима. Такође, *replay* заштита би се могла проширити употребом дијеленог складишта, као што је *Redis*, како би механизам био погодан за више инстанци истог сервиса. Додатно, могуће је проширити библиотеку подршком за ротацију кључева, детаљније политике ауторизације и интеграцију са TLS/mTLS механизмима. Као истраживачки правац остаје и поређење *Schnorr* приступа са сложенијим ZKP системима, као што су *zk-SNARK* протоколи, у погледу перформанси, сложености имплементације и погодности за микросервисна окружења.

6 Литература

- [1] Sam Newman, *Building Microservices*, O'Reilly, 2021.
- [2] Chris Richardson, *Microservices Patterns*, Manning, 2018.
- [3] Eric Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, 2018.
- [4] J. Humble et al., *A Simpler Approach to Inter-Application Authentication Utilising Cloud-Native Identities*, Journal of Cloud Computing, 2026.
- [5] Goldwasser, Micali, Rackoff, *The Knowledge Complexity of Interactive Proof Systems*, SIAM Journal on Computing, 1989. DOI: 10.1137/0218012
- [6] L. Chuat, S. Plocher, A. Perrig, “Zero-Knowledge User Authentication: An Old Idea Whose Time Has Come,” *International Workshop on Security Protocols (SPW 2019)*, 2019. DOI: 10.1007/978-3-030-57043-9_19.
- [7] C. P. Schnorr, “Efficient Identification and Signatures for Smart Cards,” *Advances in Cryptology — CRYPTO '89*, Springer, 1990, pp. 239–252.
- [8] OWASP Foundation, *REST Security Cheat Sheet*.
- [9] Jones, Bradley, Sakimura, *JSON Web Token (JWT)*, RFC 7519, 2015.
- [10] Chandramouli, Butcher, Chetal, *Attribute-based Access Control for Microservices-based Applications using a Service Mesh*, NIST SP 800-204B, 2021.
- [11] Dwivedi et al., *Privacy Preserving Authentication System based on Non-interactive Zero Knowledge Proof Suitable for Internet of Things*, Journal of Ambient Intelligence and Humanized Computing, 2021.
- [12] M. Blum, P. Feldman, S. Micali, “Non-Interactive Zero-Knowledge and Its Applications,” *Proceedings of the 20th Annual ACM Symposium on Theory of Computing (STOC '88)*, 1988, pp. 103–112. DOI: 10.1145/62212.62222.
- [13] A. Fiat, A. Shamir, “How to Prove Yourself: Practical Solutions to Identification and Signature Problems,” *Advances in Cryptology — CRYPTO '86*, Lecture Notes in Computer Science, vol. 263, Springer, 1987, pp. 186–194.
- [14] E. Ben-Sasson et al., “Zerocash: Decentralized Anonymous Payments from Bitcoin,” *2014 IEEE Symposium on Security and Privacy*, 2014, pp. 459–474. DOI: 10.1109/SP.2014.36.
- [15] J. Camenisch, A. Lysyanskaya, “An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation,” *Advances in Cryptology — EUROCRYPT 2001*, Lecture Notes in Computer Science, vol. 2045, Springer, 2001, pp. 93–118.
- [16] R. Cramer, R. Gennaro, B. Schoenmakers, “A Secure and Optimally Efficient Multi-Authority Election Scheme,” *Advances in Cryptology — EUROCRYPT '97*, Lecture Notes in Computer Science, vol. 1233, Springer, 1997, pp. 103–118.

7 Биографија

Наташа Кашиковић је рођена 5. фебруара 2003. године у Требињу. Основну школу „Јован Јовановић Змај” завршила је у родном граду. Након тога уписала је Гимназију „Јован Дучић” у Требињу, општи смјер, коју је завршила 2022. године као носилац Вукове дипломе. Исте године уписала је Факултет техничких наука Универзитета у Новом Саду, студијски програм Софтверско инжењерство и информационе технологије. Током студија остварила је висок академски успјех. Поред редовних студијских обавеза, учествовала је на семинару из области рачунарских наука у Истраживачкој станици Петница, као и на хакатонима. Стручно искуство стицала је кроз праксу у компанији Schneider Electric.